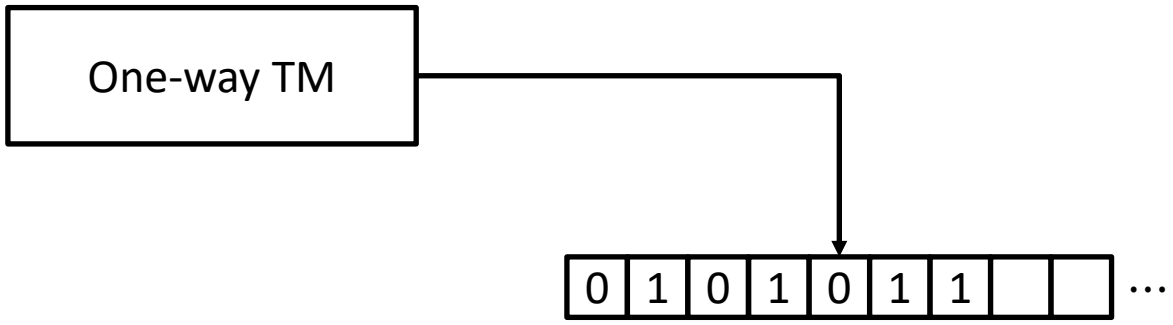
The Theory of Computing

Variants of Turing Machines

There are variants of Turing machines like multi-tape, non-deterministic, probabilistic, and quantum Turing machines. Which, if any, is the most powerful?

(Spoilers: The all have the same computational power.)

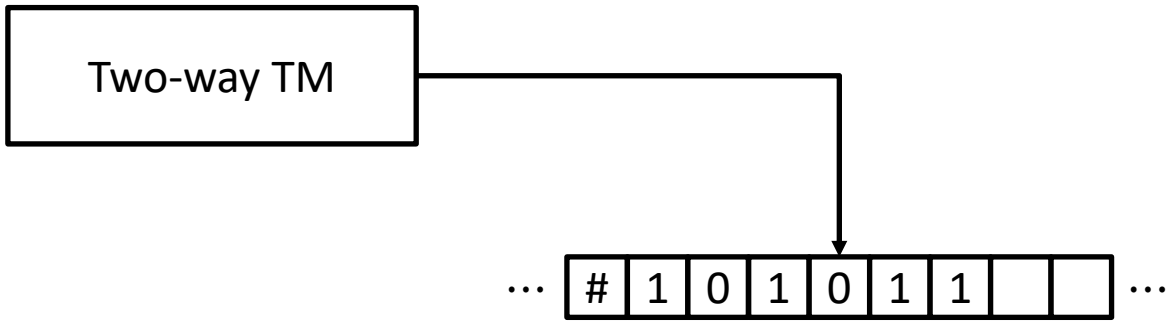


Definition: A one-way infinite Turing machine (1TM) is a variant of the deterministic Turing machine where the tape is infinite only in one direction (to the right). The machine rejects if the tape head moves left from the left most cell.

At first glance a two-way infinite tape seems more powerful than a one-way infinite tape.

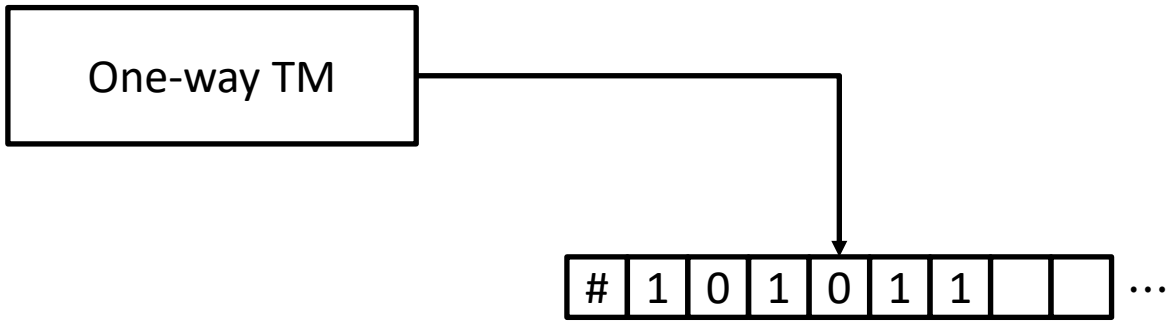
However, there are only two functional differences between the models:

1. A two-way infinite Turing machine doesn't reject when it moves left from the first cell.
2. A one-way infinite Turing machine can't move left from the left most cell.



Claim: A 2TM can simulate a 1TM.

Proof Idea: Before simulating a 1TM the 2TM will mark the cell to the left of the input with a special symbol. Should that symbol ever be scanned during simulation, the 2TM will reject, to simulate the 1TM.



Claim: A 1TM can simulate a 2TM.

Proof Idea: When the 1TM tries to move left of the left most cell, we need to create more space. For this we will use a sub-routine, that pauses the simulation, moves the whole tape contents one cell to the right, adding a blank spot in the new space, and then returning to simulation. A special symbol will be used to mark the first cell of the tape, so we never move left from it.

The 1TM and the 2TM have equivalent computational power.

Definition: A non-deterministic Turing machine is a 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$, where Q, Σ, and Γ are finite sets and:

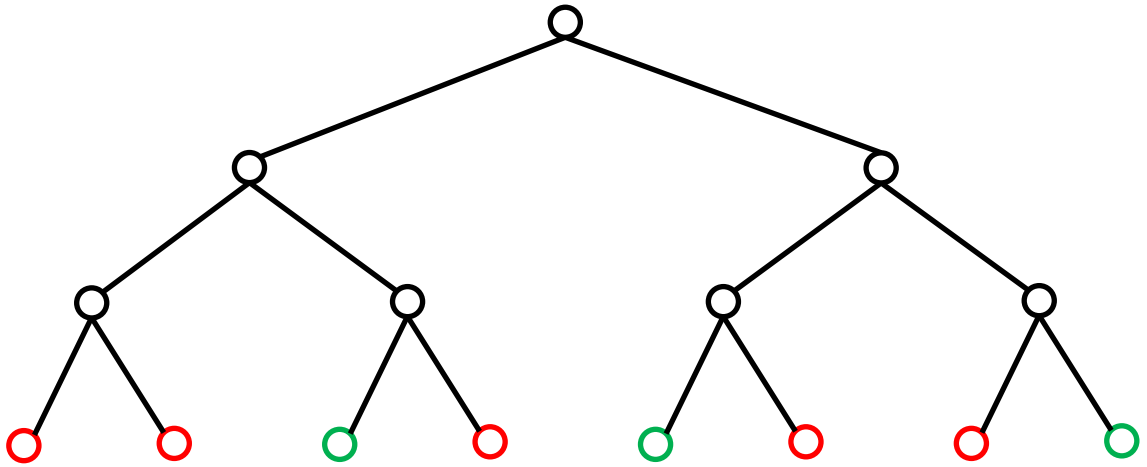
1. Q is the set of states,
2. Σ is the input alphabet not containing the blank symbol $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subset \Gamma$,
4. $\delta: Q \times \Gamma \to P(Q \times \Gamma \times \{L, R\})$is the transition function,
5. $q_0 \in Q$ is the start state,
6. $q_a \in Q$ is the accept state, and
7. $q_r \in Q$ is the reject state where $q_r \neq q_a$.

Definition: A non-deterministic Turing machine is a 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$, where Q, Σ, and Γ are finite sets and:

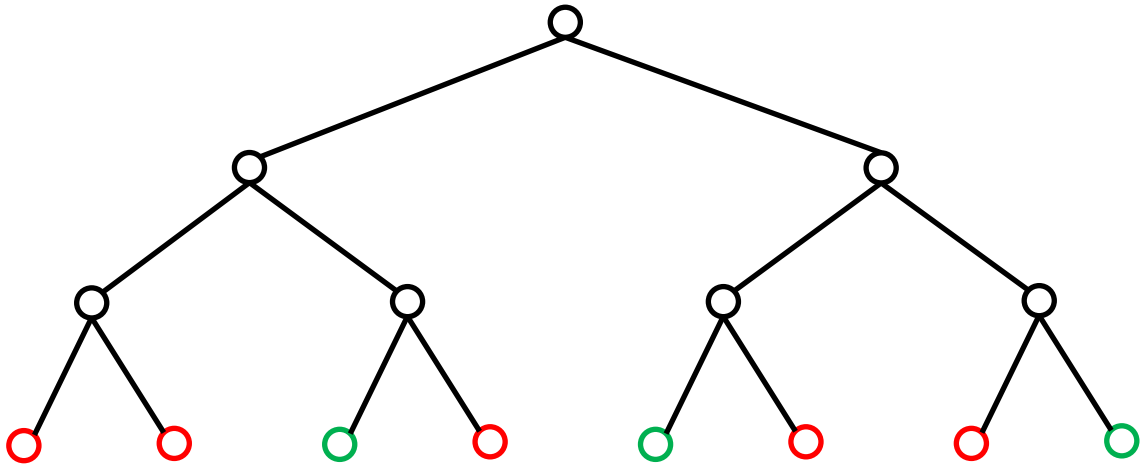
1. Q is the set of states,
2. Σ is the input alphabet not containing the blank symbol $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subset \Gamma$,
4. $\delta: Q \times \Gamma \to P(Q \times \Gamma \times \{L, R\})$ is the transition function,
5. $q_0 \in Q$ is the start state,
6. $q_a \in Q$ is the accept state, and
7. $q_r \in Q$ is the reject state where $q_r \neq q_a$.

$$q_0 0011 \Rightarrow q_1 011 \Rightarrow 0 q_1 11 \Rightarrow 01 q_1 1 \Rightarrow 011 q_1 \Rightarrow 01 q_2 1 \Rightarrow 0 q_3 1 \Rightarrow q_3 01$$
$$\Rightarrow q_3 _01 \Rightarrow q_0 01 \Rightarrow q_1 1 \Rightarrow 1 q_1 \Rightarrow q_2 1 \Rightarrow q_3 \Rightarrow q_0 \Rightarrow q_a$$

In a deterministic Turing machine, a computation is a single sequence of configurations.



In a non-deterministic Turing machine, a computation is a tree of configurations.

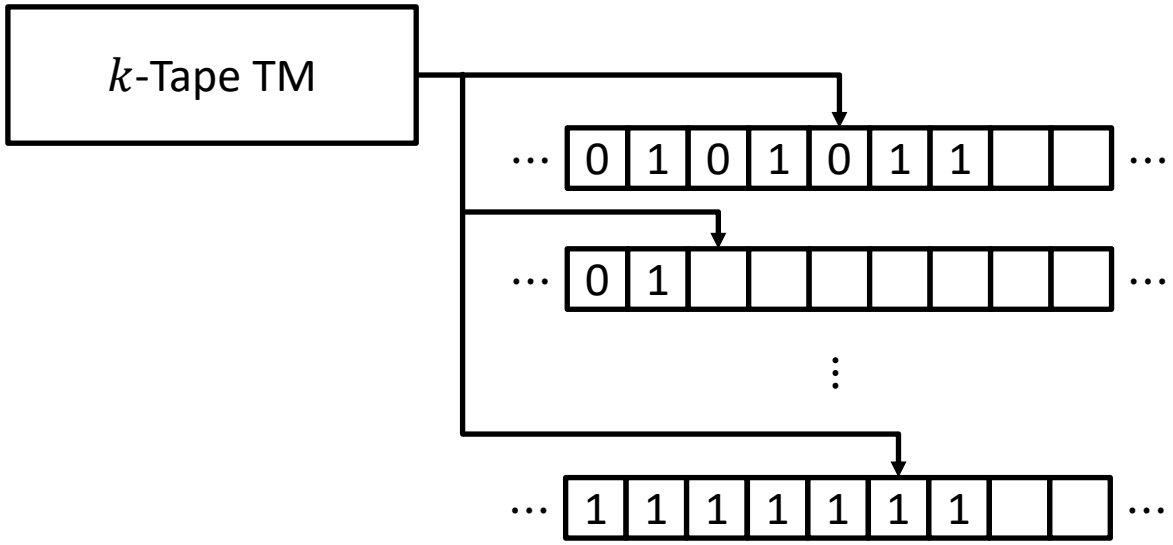


Definition: A non-deterministic Turing machine accepts a string w if any configuration of the computation tree is an accepting configuration.

We want to show a non-deterministic Turing machine (NTM) is of equivalent power to a deterministic Turing machine (DTM).

Every DTM is an NTM, so we must show that an NTM can be simulated by a DTM.

As an intermediate model for use in our proof we will define a k-tape Turing machine (kTM).



Definition: A k-tape Turing machine (k-TM) is a Turing machine with k two-way infinite tapes.

We will show a NTM can be simulated by a kTM, and that a kTM can be simulated by a DTM, showing that an NTM can be simulated by a DTM via transitivity.

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea: Consider an NTM, M_1.

We will simulate M_1 by conducting a breadth-first search on the computation tree of M_1 with a three-tape DTM, M_2.

On tape one we will store the input string and never alter it. We will copy this string to tape two when needed.

On tape two we will simulate M_1 along one branch of its computation tree.

On tape three we will store a string to keep track of which non-deterministic branch we are currently simulating.

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea: Consider an NTM, M_1.

We will simulate M_1 by conducting a breadth-first search on the computation tree of M_1 with a three-tape DTM, M_2.

On tape one we will store the input string and never alter it. We will copy this string to tape two when needed.

On tape two we will simulate M_1 along one branch of its computation tree.

On tape three we will store a string to keep track of which non-deterministic branch we are currently simulating.

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea: Consider an NTM, M_1.

We will simulate M_1 by conducting a breadth-first search on the computation tree of M_1 with a three-tape DTM, M_2.

On tape one we will store the input string and never alter it. We will copy this string to tape two when needed.

On tape two we will simulate M_1 along one branch of its computation tree.

On tape three we will store a string to keep track of which non-deterministic branch we are currently simulating.

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea: Consider an NTM, M_1.

We will simulate M_1 by conducting a breadth-first search on the computation tree of M_1 with a three-tape DTM, M_2.

On tape one we will store the input string and never alter it. We will copy this string to tape two when needed.

On tape two we will simulate M_1 along one branch of its computation tree.

On tape three we will store a string to keep track of which non-deterministic branch we are currently simulating.

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea (continued): We will use tape three to keep track of which non-deterministic branch we are currently simulating.

We do this by writing a string on tape three using the alphabet $\Gamma_b = \{1,2,\dots,b\}$ where b is the maximum branching factor of M_1.

The string will code d non-deterministic choices with d characters.

When simulating a non-deterministic choice of M_1, M_2 will consult the next character on tape three and select the non-deterministic transition encoded by the character.

If no such character or transition exists, simulation of this branch ends.

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea (continued): We will use tape three to keep track of which non-deterministic branch we are currently simulating.

We do this by writing a string on tape three using the alphabet $\Gamma_b = \{1,2,\dots,b\}$ where b is the maximum branching factor of M_1.

The string will code d non-deterministic choices with d characters.

When simulating a non-deterministic choice of M_1, M_2 will consult the next character on tape three and select the non-deterministic transition encoded by the character.

If no such character or transition exists, simulation of this branch ends.

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea (continued): We will use tape three to keep track of which non-deterministic branch we are currently simulating.

We do this by writing a string on tape three using the alphabet $\Gamma_b = \{1,2,\dots,b\}$ where b is the maximum branching factor of M_1.

The string will code d non-deterministic choices with d characters.

When simulating a non-deterministic choice of M_1, M_2 will consult the next character on tape three and select the non-deterministic transition encoded by the character.

If no such character or transition exists, simulation of this branch ends.

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea (continued): We simulate each branch to depth d (beginning with $d = 1$) in this way:

1. Copy the input string from tape one to tape two.
2. Write the first string of length d over the alphabet Γ_b on tape three.
3. Simulate M_1 d steps on tape two using the string on tape three to determine non-deterministic choices as described above. If an accepting configuration is reached, accept.
4. Otherwise, erase the contents of tape two and write the next string of length d over the alphabet Γ_b on tape three.
5. Repeat step 3-4 for every string of length d over the alphabet Γ_b.
6. If no accepting configuration is found, we repeat steps 1-6 for depth $d + 1$. □

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea (continued): We simulate each branch to depth d (beginning with $d = 1$) in this way:

1. Copy the input string from tape one to tape two.
2. Write the first string of length d over the alphabet Γ_b on tape three.
3. Simulate M_1 d steps on tape two using the string on tape three to determine non-deterministic choices as described above. If an accepting configuration is reached, accept.
4. Otherwise, erase the contents of tape two and write the next string of length d over the alphabet Γ_b on tape three.
5. Repeat step 3-4 for every string of length d over the alphabet Γ_b.
6. If no accepting configuration is found, we repeat steps 1-6 for depth $d + 1$. □

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea (continued): We simulate each branch to depth d (beginning with $d = 1$) in this way:

1. Copy the input string from tape one to tape two.
2. Write the first string of length d over the alphabet Γ_b on tape three.
3. Simulate M_1 d steps on tape two using the string on tape three to determine non-deterministic choices as described above. If an accepting configuration is reached, accept.
4. Otherwise, erase the contents of tape two and write the next string of length d over the alphabet Γ_b on tape three.
5. Repeat step 3-4 for every string of length d over the alphabet Γ_b.
6. If no accepting configuration is found, we repeat steps 1-6 for depth $d + 1$. □

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea (continued): We simulate each branch to depth d (beginning with $d = 1$) in this way:

1. Copy the input string from tape one to tape two.
2. Write the first string of length d over the alphabet Γ_b on tape three.
3. Simulate M_1 d steps on tape two using the string on tape three to determine non-deterministic choices as described above. If an accepting configuration is reached, accept.
4. Otherwise, erase the contents of tape two and write the next string of length d over the alphabet Γ_b on tape three.
5. Repeat step 3-4 for every string of length d over the alphabet Γ_b.
6. If no accepting configuration is found, we repeat steps 1-6 for depth $d + 1$. □

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea (continued): We simulate each branch to depth d (beginning with $d = 1$) in this way:

1. Copy the input string from tape one to tape two.
2. Write the first string of length d over the alphabet Γ_b on tape three.
3. Simulate M_1 d steps on tape two using the string on tape three to determine non-deterministic choices as described above. If an accepting configuration is reached, accept.
4. Otherwise, erase the contents of tape two and write the next string of length d over the alphabet Γ_b on tape three.
5. Repeat step 3-4 for every string of length d over the alphabet Γ_b.
6. If no accepting configuration is found, we repeat steps 1-6 for depth $d + 1$. □

Theorem: A non-deterministic Turing machine (NTM) can be simulated by a k-tape Turing machine (kTM).

Proof idea (continued): We simulate each branch to depth d (beginning with $d = 1$) in this way:

1. Copy the input string from tape one to tape two.
2. Write the first string of length d over the alphabet Γ_b on tape three.
3. Simulate M_1 d steps on tape two using the string on tape three to determine non-deterministic choices as described above. If an accepting configuration is reached, accept.
4. Otherwise, erase the contents of tape two and write the next string of length d over the alphabet Γ_b on tape three.
5. Repeat step 3-4 for every string of length d over the alphabet Γ_b.
6. If no accepting configuration is found, we repeat steps 1-6 for depth $d + 1$. □

So, an NTM can be simulated by a 3TM, but can that 3TM be simulated by a DTM, completing our proof?

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea: Let M_1 be a k-tape Turing machine.

We will simulate M_1 with a single-tape Turing machine M_2.

We need to simulate k tapes on a single tape.

This means we must store all the contents of the k tapes and must remember the locations of the k tape heads.

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea: Let M_1 be a k-tape Turing machine.

We will simulate M_1 with a single-tape Turing machine M_2.

We need to simulate k tapes on a single tape.

This means we must store all the contents of the k tapes and must remember the locations of the k tape heads.

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea (continued): This means we must store all the contents of the k tapes and must remember the locations of the k tape heads.

We will store the contents of the k tapes on the single tape, sequentially, separated by special # symbols that are not in the tape alphabet of M_1.

Initially, the input is written on the first tape, and all others are empty.

We will remember the location of each tape head by marking the tape cell the head is scanning. For instance, we can mark a 0 with a dot $\dot{0}$.

If during simulation any tape head moves outside of the simulated tape contents, we use the subroutine from the last construction to move all symbols to the right to create more space before continuing simulation.

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea (continued): This means we must store all the contents of the k tapes and must remember the locations of the k tape heads.

We will store the contents of the k tapes on the single tape, sequentially, separated by special # symbols that are not in the tape alphabet of M_1.

Initially, the input is written on the first tape, and all others are empty.

We will remember the location of each tape head by marking the tape cell the head is scanning. For instance, we can mark a 0 with a dot $\dot{0}$.

If during simulation any tape head moves outside of the simulated tape contents, we use the subroutine from the last construction to move all symbols to the right to create more space before continuing simulation.

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea (continued): This means we must store all the contents of the k tapes and must remember the locations of the k tape heads.

We will store the contents of the k tapes on the single tape, sequentially, separated by special # symbols that are not in the tape alphabet of M_1.

Initially, the input is written on the first tape, and all others are empty.

We will remember the location of each tape head by marking the tape cell the head is scanning. For instance, we can mark a 0 with a dot $\dot{0}$.

If during simulation any tape head moves outside of the simulated tape contents, we use the subroutine from the last construction to move all symbols to the right to create more space before continuing simulation.

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea (continued): This means we must store all the contents of the k tapes and must remember the locations of the k tape heads.

We will store the contents of the k tapes on the single tape, sequentially, separated by special # symbols that are not in the tape alphabet of M_1.

Initially, the input is written on the first tape, and all others are empty.

We will remember the location of each tape head by marking the tape cell the head is scanning. For instance, we can mark a 0 with a dot $\dot{0}$.

If during simulation any tape head moves outside of the simulated tape contents, we use the subroutine from the last construction to move all symbols to the right to create more space before continuing simulation.

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea (continued):

At each transition we must update all k tape contents.

First, we scan to each tape head position and remember the symbols scanned.

Then we return to each tape head position and make the appropriate change to the symbol and move the tape head.

If the machine enters a halting configuration, we accept or reject as it does. □

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea (continued):

At each transition we must update all k tape contents.

First, we scan to each tape head position and remember the symbols scanned.

Then we return to each tape head position and make the appropriate change to the symbol and move the tape head.

If the machine enters a halting configuration, we accept or reject as it does. □

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea (continued):

At each transition we must update all k tape contents.

First, we scan to each tape head position and remember the symbols scanned.

Then we return to each tape head position and make the appropriate change to the symbol and move the tape head.

If the machine enters a halting configuration, we accept or reject as it does. □

Theorem: A k-tape Turing machine can be simulated by a single-tape Turing machine.

Proof idea (continued):

At each transition we must update all k tape contents.

First, we scan to each tape head position and remember the symbols scanned.

Then we return to each tape head position and make the appropriate change to the symbol and move the tape head.

If the machine enters a halting configuration, we accept or reject as it does. □

Non-deterministic Turing machines are of equivalent computational power as deterministic Turing machines.

However, when a deterministic Turing machine simulates a non-deterministic Turing machine using this method it takes exponentially longer.

It is unknown if this is necessary.

This question is central to the P = NP open question in complexity theory.

The following variants of Turing machines are equivalent in computational power to a single-tape deterministic Turing machine:

- multi-tape Turing machine,
- multi-head Turing machine,
- non-deterministic Turing machine,
- probabilistic Turing machine, and
- quantum Turing machine.

Definition: A model of computation is Turing-complete if it is equivalent in computational power to a Turing machine.

The Theory of Computing

The Church-Turing Thesis

We can use Turing machines to compute mathematical functions by treating the contents of the tape after computation halts as the output.

This also requires us to use an encoding of natural numbers as strings using some number base.

For simplicity, we have been using unary so far.

$$\epsilon, 0, 00, 000, \ldots$$

represents

$$0, 1, 2, 3, \ldots$$

We could also use another base like binary.

$$0, 1, 10, 11, \ldots$$

represents

$$0, 1, 2, 3, \ldots$$

Definition: A function $f: \mathbb{N} \to \mathbb{N}$ is Turing computable if and only there exists a Turing machine that halts on every input string encoding a number n with $f(n)$ written on the tape.



Alonzo Church was an American mathematician and contemporary of Alan Turing.



Church created a functional model of computation called the lambda calculus.

An anonymously defined function is a function without a name.

$$f(n) = n^2$$

$$(n) \Rightarrow n^2$$

In the lambda calculus, anonymous functions can be applied to arguments like this.

$$\big((n) \Rightarrow n^2\big)(5) = 25$$

In the lambda calculus, anonymous functions can be applied to other functions.

$$\big((n) \Rightarrow n^2\big)\big((m) \Rightarrow m + 1\big) = \big((m) \Rightarrow (m + 1)^2\big)$$

Church was interested in the kinds of functions he could compute with his lambda calculus beginning with a very simple initial set of definitions.

$$0$$
$$(n) \Rightarrow n + 1$$

With these two functions we can define the natural numbers.

$$1 = \big((n) \Rightarrow n + 1\big)(0)$$
$$2 = \big((n) \Rightarrow n + 1\big)(1)$$
$$3 = \big((n) \Rightarrow n + 1\big)(2)$$
$$\vdots$$

$$0$$
$$(n) \Rightarrow n + 1$$

If we expand one of these, we can define other functions.

$$1 = \big((n) \Rightarrow n + 1\big)(0)$$
$$2 = \big((n) \Rightarrow n + 1\big)\big((n) \Rightarrow n + 1\big)(0) = \big((n) \Rightarrow n + 2\big)(0)$$
$$3 = \big((n) \Rightarrow n + 1\big)\big((n) \Rightarrow n + 1\big)\big((n) \Rightarrow n + 1\big)(0) = \big((n) \Rightarrow n + 3\big)(0)$$
$$\vdots$$

This function returns another function.

$$\Big((n) \Rightarrow \big((m) \Rightarrow n + m\big)\Big)$$

If we call it once, we still have a function.

$$\Big((n) \Rightarrow \big((m) \Rightarrow (n+m)\big)\Big)(5) = \big((m) \Rightarrow (5+m)\big)$$

But if we call it twice, we get a number.

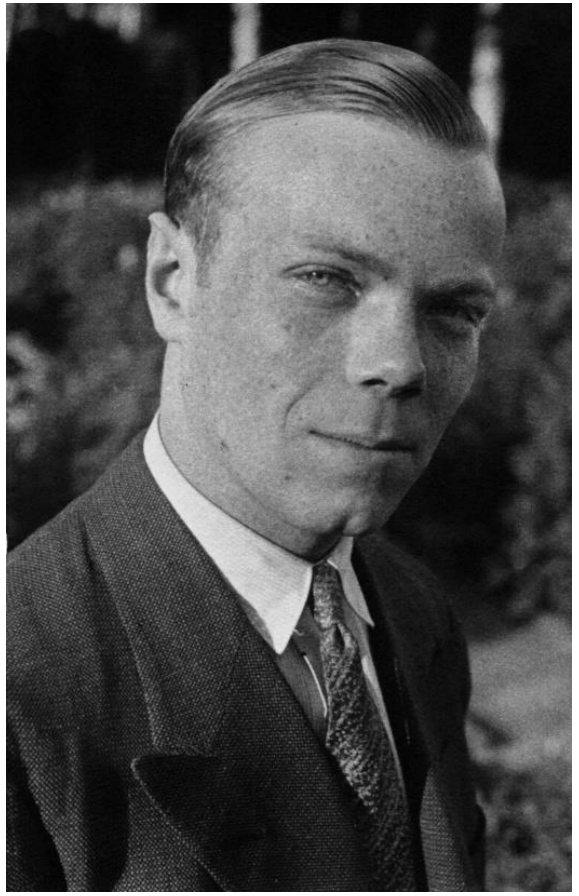
$$\Big((n) \Rightarrow \big((m) \Rightarrow (n+m)\big)\Big)(5)(4)$$
$$= \big((m) \Rightarrow (5+m)\big)(4)$$
$$= 5 + 4$$
$$= 9$$

We can use a shorthand to define functions with more than one argument.

$$\big((n,m) \Rightarrow (n+m)\big)(5,4)$$
$$= 5 + 4$$
$$= 9$$

Using functions like these Church defined a set of mathematical functions he called computable functions.





A third set of functions, the general recursive functions, were shown by Kurt Gödel and Jacques Herbrand to be computable using recursion, composition, and minimization.

Turing, Church and Kleene proved that these three classes of functions coincided, or rather, that the three models of computation were of equivalent power.

The Church-Turing Hypothesis: Any physically realizable model of computation is computationally equivalent to the Turing machine.

These formal models of computation are known to be Turing-complete:

- the lambda calculus
- primitive recursion
- Conway's Game of Life
- Wolfram's elementary cellular automata Rule 110

These practical computing devices are Turing-complete:

- the Von Neumann computer architecture
- all general-purpose programming languages (e.g., Python, Java, C/C++/C#, JavaScript)
- many specialized programming languages (e.g., SQL, TeX, Excel)

Some software systems are unintentionally Turing-complete:

- Minesweeper (with an infinite grid)
- Minecraft
- Magic: The Gathering
- Dwarf Fortress

The Theory of Computing

Decidable Languages

$$L_1 = \left\{0^n \# 0^{n^2} \mid n \geq 0\right\}$$

So far, we have used simple encodings to ask questions about numbers.

$$L_1 = \left\{0^n \# 0^{n^2} | n \geq 0\right\}$$

In this language, we encode numbers in unary using only zeros.

$$L_2 = \{w\#x \mid w \text{ and } x \text{ are binary numbers such that } x = w^2\}$$

In this similar language, we encode the numbers in binary instead.

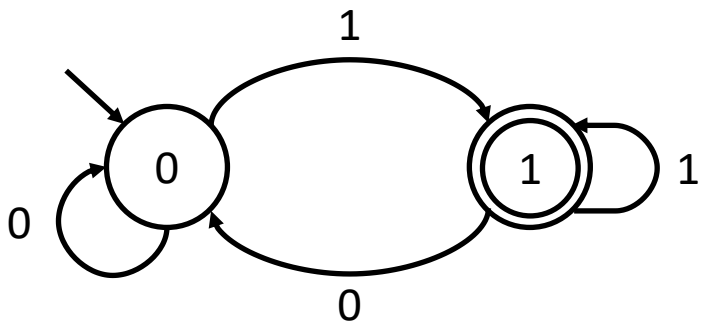
In these examples, the encoding scheme is explicitly stated, but often we will leave the encoding implicit.

$$L_3 = \{\langle n, m\rangle | \ n \text{ and } m \text{ are numbers such that } n = m^2\}$$

In this similar language, we use angle braces $\langle E \rangle$ to indicate that the input has been encoded but do not state the encoding scheme.

$$L_4 = \{\langle S, s\rangle |\ S \text{ is a set and } s \in S\}$$
$$L_5 = \{\langle G\rangle |\ G \text{ is a connected graph}\}$$
$$L_6 = \{\langle D, w\rangle |\ D \text{ is a DFA and } D \text{ accepts } w\}$$

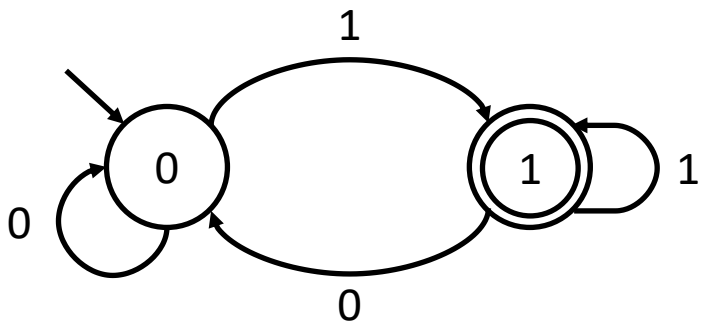
In these languages, we encode different kinds of inputs, and in general we assume any well-defined mathematical object can be encoded.



Consider this deterministic finite automaton formalized as $D = \{Q, \Sigma, \delta, q_0, F\}$.

We can encode this machine as one long string over a complex alphabet:

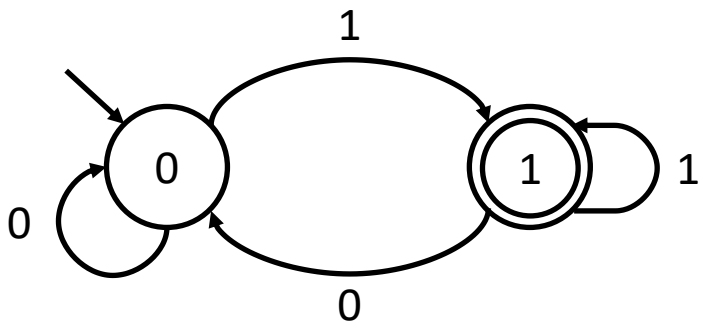
$$\{\{q_0, q_1\}, \{\sigma_0, \sigma_1\}, \delta(q_0, \sigma_0) = q_0, \delta(q_0, \sigma_1) = q_1, \delta(q_1, \sigma_0) = q_0, \delta(q_1, \sigma_1) = q_1, q_0, \{q_1\}\}$$



$$\{\{q_0, q_1\}, \{\sigma_0, \sigma_1\}, \delta(q_0, \sigma_0) = q_0, \delta(q_0, \sigma_1) = q_1, \delta(q_1, \sigma_0) = q_0, \delta(q_1, \sigma_1) = q_1, q_0, \{q_1\}\}$$

The alphabet we use for this string is:

$$\Gamma = \{\{, \}, (,), q, \sigma, \delta, 0, 1, , \}$$



$$\Gamma = \{\{, \}, (,), q, \sigma, \delta, 0, 1, , , \}$$

Any alphabet can be further encoded into binary (as needed) using a set of code words like these:

$$0 = 0000, 1 = 1111$$
$$\} = 0001, \{ = 0010,) = 0011, (= 0100$$
$$q = 0101, \sigma = 0110, \delta = 0111, , = 1000$$

So, we assume any well-defined mathematical object can be written as a string of symbols, and these symbols can ultimately be encoded in any alphabet as needed.

In addition, any such encoded object can be decoded by a Turing machine.

Some encoding schemes create a one-to-one and onto relationship between objects and strings, but this is not often the case.

If an input string does not properly encode an expected input type, we assume the string is rejected by any Turing machine processing it.

$$A_{DFA} = \{\langle D, w\rangle \mid D \text{ is a DFA that accepts the string } w\}$$

$$A_{DFA} = \{\langle D, w\rangle \mid D \text{ is a DFA that accepts the string } w\}$$

Theorem: A_{DFA} is a decidable language.

Proof idea: We will construct a TM M_1 that decides A_{DFA}.

Let M_1 = "On input $\langle D, w\rangle$, where D is a DFA and w is a string:

1. Write the initial state q_0 on tape two.
2. Scan the current symbol and current state and check the transition function for the next state.
3. Update the current state and move to the next input symbol.
4. Repeat step 2 and 3 until all symbols are used.
5. If the simulation ends in an accept state, accept.
6. Reject."

We can reuse this M_1 machine to show that other languages are also decidable.

$$A_{NFA} = \{\langle N, w\rangle \mid N \text{ is a NFA that accepts the string } w\}$$

Theorem: A_{NFA} is a decidable language.

Proof idea: We will construct a TM M_2 that decides A_{NFA}.

Let M_2 = "On input $\langle N, w\rangle$, where N is a NFA and w is a string:

1. Convert the NFA N into a DFA D.
2. Simulate machine M_1 on input $\langle D, w\rangle$.
3. If the simulation ends in an accept state, accept.
4. Reject."

$$A_{RE} = \{\langle R, w\rangle \mid R \text{ is a RE that accepts the string } w\}$$

Theorem: A_{RE} is a decidable language.

Proof idea: We will construct a TM M_3 that decides A_{RE}.

Let M_3 = "On input $\langle R, w\rangle$, where R is a RE and w is a string:

1. Convert the RE R into a NFA N.
2. Simulate machine M_2 on input $\langle N, w\rangle$.
3. If the simulation ends in an accept state, accept.
4. Reject."

$$E_{DFA} = \{\langle D \rangle \mid D \text{ is a DFA that accepts no strings}\}$$

Theorem: E_{DFA} is a decidable language.

Proof idea: We will construct a TM M_4 that decides E_{DFA}.

Let M_4 = "On input $\langle D \rangle$, where D is a DFA:

1. Mark the start state.
2. Mark any state with an incoming transition from a marked state.
3. Repeat step two until no new state is marked.
4. If an accept state is marked, reject.
5. Accept."

$$EQ_{DFA} = \{\langle D_1, D_2 \rangle \mid D_1 \text{ and } D_2 \text{ are DFAs such that } L(D_1) = L(D_2)\}$$

Theorem: EQ_{DFA} is a decidable language.

Proof idea: We will construct a TM M_5 that decides EQ_{DFA}.

Let M_5 = "On input $\langle D_1, D_2 \rangle$, where D_1 and D_2 are DFAs:

1. Construct a DFA D_3 such that:
$$L(D_3) = \left(L(D_1) \cap \overline{L(D_2)}\right) \cup \left(\overline{L(D_1)} \cap L(D_2)\right)$$
2. Simulate machine M_4 on input $\langle D_3 \rangle$.
3. If the simulation ends in an accept state, accept.
4. Reject."

$$A_{CFG} = \{\langle G, w\rangle \mid G \text{ is a CFG that generates the string } w\}$$

Theorem: A_{CFG} is a decidable language.

Proof idea: We will construct a TM M_6 that decides A_{CFG}.

Note that any derivation of w from G if G is in Chomsky normal form has $2n - 1$ steps where n is the length of w.

Let $M_6 =$ "On input $\langle G, w\rangle$, where G is a CFG and w is a string:

1. Convert G to Chomsky normal form.
2. If $n = 0$ list all derivations with one step.
3. Otherwise, list all derivations with $2n - 1$ steps where n is the length of w.
4. If any derivation generates w, accept.
5. Reject."

$$E_{CFG} = \{\langle G \rangle \mid G \text{ is a CFG such that } G \text{ generates no strings}\}$$

Theorem: E_{CFG} is a decidable language.

Proof idea: We will construct a TM M_7 that decides E_{CFG}.

Let M_7 ="On input $\langle G \rangle$, where G is a CFG:

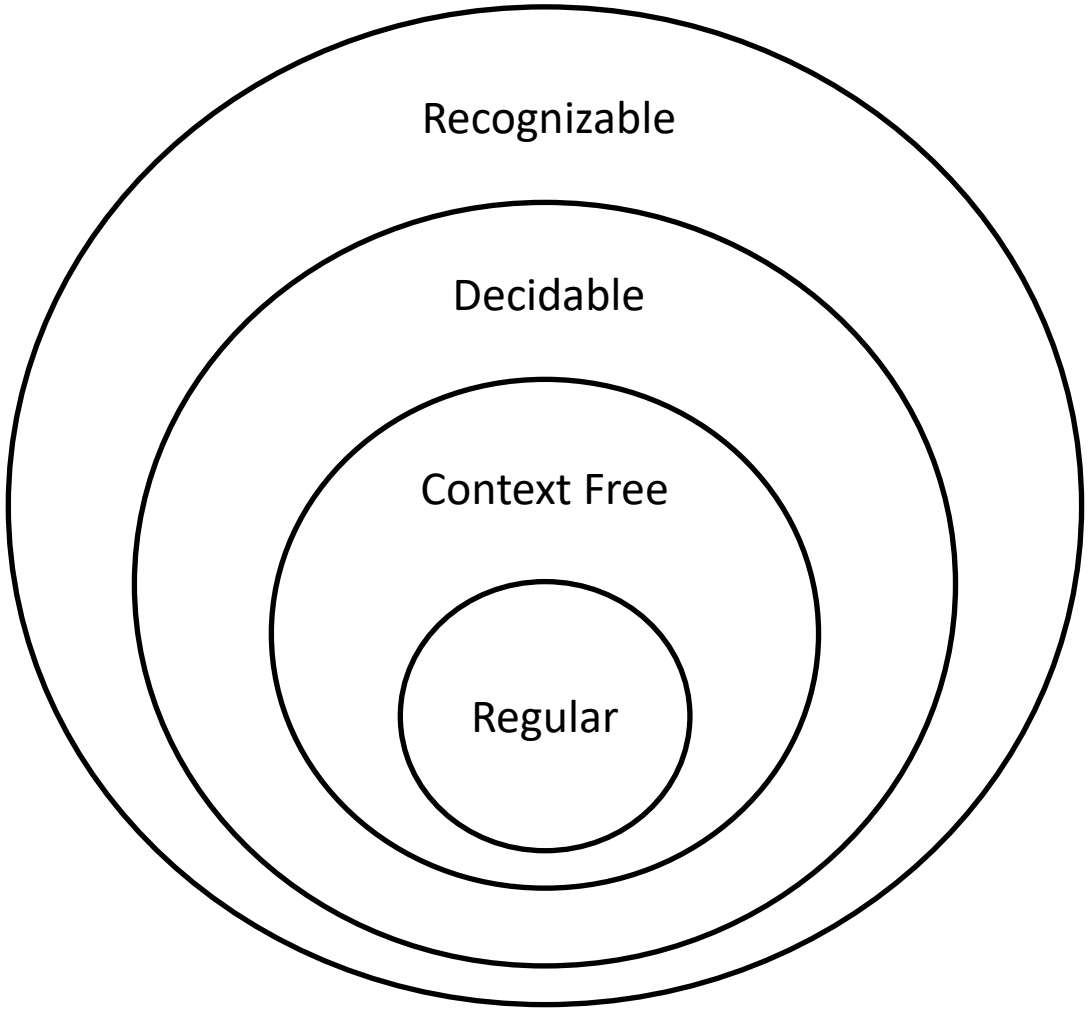
1. Mark all terminal symbols in G.
2. Mark a variables A where G has a rule $A \to U_1 U_2 \cdots U_k$ and each symbol $U_1, U_2, \ldots, U_k$ has already been marked. Repeat this step until all such variables have been marked.
3. If the start symbol is not marked, accept.
4. Reject."

Theorem: Every context free language is a decidable language.

Proof idea: Let L be a context free language and let G be a context free grammar such that $L(G) = L$. We will construct a TM M_G that decides L.

Let $M_6 =$ "On input w:

1. Write an encoding of G on the tape.
2. Simulate machine M_6 on input $\langle G, w \rangle$.
3. If the simulation ends in an accept state, accept.
4. Reject."



Theorem: Every context free language is a decidable language.